Deep Learning Frameworks and Optimization

Deep Learning · Unit 2 · Exam Prep Notes (easy language)
Based on the Unit 2 slides (Dr. P Sree Lakshmi). Topics: TensorFlow 2.x, PyTorch, Gradient Clipping, Learning Rate Scheduling, Optimizers (Adam), Dropout, Batch Normalization, Distributed Training on GPUs/TPUs.

> **Unit 2 at a glance:** Frameworks (TensorFlow vs PyTorch) → Gradient clipping (stop exploding gradients) → Learning rate scheduling → Optimizers (GD → SGD → Momentum → AdaGrad → RMSprop → Adam) → Regularization (Dropout, Batch Norm) → Distributed training (GPU/TPU).

1. Deep Learning Frameworks

- A **framework** makes it easy to build and train neural networks by giving ready-made components, optimization functions and management tools. It gives high-level interfaces, advanced optimizers, and support for big datasets and complex models.

Framework	By	Key point
TensorFlow	Google	Powerful and flexible; runs on cloud servers to mobile; good for research and large-scale projects
PyTorch	Facebook (Meta)	User-friendly, flexible, easy to debug; ideal for research and experiments
Keras	Part of TensorFlow	Simple way to build and test models quickly
MXNet	Apache	Efficient; handles large big-data projects
Caffe	Berkeley	Optimized for speed, especially image classification
JAX	Google	Newer; high performance on GPUs/TPUs; popular in research

1.1 TensorFlow 2.x

Developed by Google. Makes building, training and deploying neural networks simple, fast and flexible. Much more user-friendly than TensorFlow 1.x.

Key feature	Meaning
1. **Eager execution**	Code runs line by line like normal Python. (TF 1.x: first build a graph, then run it.) Easy to debug.
2. **Keras as default API**	`tf.keras` is built in: Sequential models, Functional API, pretrained models.
3. Clean, Pythonic syntax	Easy to learn; looks like Python and PyTorch.
4. **Excellent deployment**	**TensorFlow Serving** (servers), **TensorFlow Lite** (mobile and IoT), **TensorFlow.js** (browsers).
5. **AutoGraph**	Converts Python code into optimized computational graphs: speed of graphs + simplicity of Python.
6. Pretrained models / transfer learning	**TensorFlow Hub**: image classification, NLP and generative models.
7. GPU/TPU support	Works with NVIDIA GPUs and Google TPUs for fast training.

Advantages	Limitations
Strong for production deployment; clean Keras API; runs on desktop, cloud, mobile, browser; large community; high accuracy	Slightly more complex than PyTorch for beginners; debugging is harder in static graph mode; fewer HuggingFace models than PyTorch

Used in: image recognition, NLP, mobile AI apps, web AI apps, robotics and IoT, large enterprise AI.

1.2 PyTorch

Open-source, developed by Facebook (Meta) in 2016. Widely used in research, teaching, experimentation and Generative AI. Easy to learn and Python-friendly.

- **Dynamic computation graph:** graph is built on the fly as code runs ("eager execution by default"). Easy to experiment and debug; suits complex architectures.
- **Clean Pythonic syntax:** looks like Python + NumPy.
- **Strong research support:** new architectures, GANs, Transformers, diffusion models, rapid prototyping. Many major papers use PyTorch.
- **HuggingFace integration:** most pretrained models appear first in PyTorch (about 195k models).
- **GPU acceleration** with NVIDIA GPUs using **CUDA**.
- **Modular and extensible:** custom layers, loss functions and training loops.
- **Deployment:** TorchServe (serving), PyTorch Mobile (Android/iOS), PyTorch Live (mobile prototypes). TensorFlow still leads in enterprise deployment.

Limitations: deployment used to be weak (improving); fewer industrial deployment resources than TensorFlow.

1.3 PyTorch vs TensorFlow (important table)

Factor	PyTorch	TensorFlow
Ease of use	Easy, Pythonic, beginner-friendly	Harder, more code; easier with Keras
Flexibility	Very flexible: **dynamic graph**, model can change while running	Less flexible: **static graph**, model fully defined before running
Popularity	Research, academia, experimentation	Industry and production
Community	Strong research support, good docs	Larger community, extensive ecosystem
Training time	Generally faster in many studies	Slightly slower in similar conditions
Pretrained models	~195k on HuggingFace	~13k on HuggingFace
Deployment	Improving (TorchServe, Mobile, Live)	Strongest (Serving, Lite, JS, IoT, cloud)
Accuracy	Good; improves over training	Often slightly higher, especially early in training

Choose PyTorch if: beginner, research/experiments, Generative AI (GANs, LLMs, transformers), more pretrained models.
Choose TensorFlow if: large-scale production apps, deployment on mobile/edge/IoT, industry-level deployment.

2. Gradient Clipping

Analogy: riding a bicycle downhill. If the slope is steep you go too fast and may crash, so you press the brakes. Gradient clipping is the **brake** for training.

- **Definition:** a technique that prevents **exploding gradients** by putting a **limit on how large gradients can become** during training.
- **Problem:** sometimes gradients become huge, causing unstable learning, loss becoming **NaN**, very large weight updates, and the model failing to learn. Common in **RNNs, LSTMs, deep networks, GANs and reinforcement learning**.

How it works (3 steps)

1. **Compute gradients normally** (backpropagation). Example: g = 12.5, safe limit = 5.
2. **Check if too large:** if |g| > limit, clip. (12.5 > 5, so clip.)
3. **Clip or scale down** (two ways below).

Method	Rule and example
Value-based clipping (simple)	Cap each gradient inside a range: **$g' = max(\ min(g, max_value), min_value\ )$**. Range [−5, 5], gradient 12.5 gives clipped gradient **5**. Each gradient is checked individually.
Norm-based clipping (most common)	Check the size (norm) of the whole gradient vector: A: norm **$||g|| = \sqrt{(g_1^2 + g_2^2 + ...)}$** B: if ||g|| > max_norm, scale the whole vector. C: **$g_{clipped} = g \times (max_norm / ||g||)$**. Keeps **direction**, shrinks **magnitude**.

Worked example (norm-based): g = [4, 8, 12], max_norm = 10.
Step 1: ||g|| = √(16 + 64 + 144) = √224 ≈ **14.97** (greater than 10, so clip).
Step 2: scale = 10 / 14.97 ≈ **0.668**.
Step 3: $g_{clipped}$ = [4×0.668, 8×0.668, 12×0.668] = **[2.672, 5.344, 8.016]**.

Key idea: clipping does not remove the gradient; it only reduces it when needed, so training stays stable, updates stay controlled and loss decreases smoothly.

Where it is used

- **Neural networks:** RNN/LSTM/GRU (most common), deep CNNs, MLPs, autoencoders, Transformers (BERT, GPT, ViT).
- **Reinforcement learning:** REINFORCE, Actor-Critic, A2C/A3C, PPO, DDPG, TD3, SAC.
- **Optimizers:** optimizer-independent; works with SGD, Momentum, Adam, RMSprop, Adagrad, AdamW.
- **NLP/sequence models, GANs/WGANs** (prevents generator/discriminator collapse), and any very large/deep models.

3. Learning Rate Scheduling

- **Definition:** automatically **changing (usually reducing) the learning rate over time** during training, so the model learns **fast at the beginning** and **carefully later**.
- **Why:** at the start weights are random, so large steps learn quickly. Later the model is near the optimum, and large steps can **overshoot** the minimum. Without scheduling, training may oscillate, loss may stop improving, and convergence is slow or unstable.
- **Analogy:** running downhill: run fast at the top, walk slowly near the bottom.
- **Benefits:** faster convergence, lower loss, higher accuracy, stable training. Works with any optimizer (SGD, Adam...).

Types of learning rate schedules

Type	Formula / idea	Example	When to use
1. Step decay (StepLR)	After every *step_size* epochs: **LR = LR × γ** (γ usually 0.1 or 0.5)	LR 0.1, step 10, γ 0.1: epochs 1-10 = 0.1; 11-20 = 0.01; 21-30 = 0.001	When a sharp drop is needed after some epochs
2. Exponential decay	**$LR = LR_0 \times \gamma^{epoch}$** (continuous decrease every epoch)	LR_0 0.1, γ 0.95: epoch 1 = 0.095, 2 = 0.090, 3 = 0.0855, 10 = 0.059	Smooth, gradual decay
3. Polynomial decay	**$LR = LR_0 (1 - epoch/max_epochs)^p$**	LR_0 0.1, p = 2, max 100; at epoch 50: $0.1 \times (0.5)^2 =$ **0.025**	Deep CNNs, object detection, large training cycles
4. ReduceLROnPlateau	Reduce LR **only when the loss stops improving**	If validation loss does not improve for 3 epochs, LR is reduced by 0.1 (e.g. val loss 0.40, 0.42, 0.44, 0.44 → LR reduced at epoch 8)	NLP, speech recognition, large CNNs, irregular datasets
5. Cyclical LR (CLR)	LR **goes up and down** between a lower bound (base_lr) and upper bound (max_lr) in a **triangular wave**; step size = iterations from min to max	Min 0.001, max 0.006, step 200: iterations 0-200 LR rises 0.001 to 0.006; 200-400 falls back; repeats	Helps model **jump out of sharp local minima**

Related terms

Term	Definition	Example
Epoch	One full forward + backward pass through the **entire training dataset**	1 epoch = all data seen once
Batch size	Number of training examples used in one forward/backward pass	1 million records, batch size 1,000
Iteration	Steps needed to finish one epoch	**Iterations = Total records / Batch size** = 1,000 in the example

Mini projects in the slides (all use a CNN with gradient clipping for stable training and LR scheduling for better convergence): handwritten digit recognition (bank cheques), plant leaf disease detection (smart farming), driver drowsiness detection, face mask detection, brain tumour detection from MRI.

4. Optimizers in Deep Learning

- The network predicts, calculates the **loss** (error), then must update its weights to reduce it. The goal is to minimize the **loss function J(θ)**, seen as a **loss landscape**.
- **Optimizer:** the algorithm that decides **how to adjust the parameters (θ)** to move down the loss landscape to the minimum.

- **Analogy (blindfolded on a hill):** hill = loss function; your position = weights; slope direction = gradient; steps you take = optimizer; goal = minimum loss.
- Optimizers decide: how big the step is, which direction, how fast to reach the bottom.
- **Importance: Convergence** (without optimizer, weights stay random); **Speed** (saves days of training); **Precision** (find the global minimum, not get stuck in a local minimum).
- **Gradient** = ∂Loss/∂w. It tells how loss changes and points in the direction of **steepest increase**, so we move in the **opposite** direction.

4.1 Gradient Descent (foundation)

> **Update rule:** $\theta_{new} = \theta_{old} - \eta \cdot \nabla J(\theta)$
> or $W_{new} = W_{old} - \eta \cdot (\partial L / \partial W_{old})$
> η = **learning rate** (step size); ∇J = gradient (slope); W = weights.

4.2 Types of gradient descent

Type	How it works	Problem / result
Batch GD (BGD)	Uses the **entire dataset** in one go to compute the gradient	Needs huge RAM, slow; impractical for large data
Stochastic GD (SGD)	Passes **one record at a time**	Solves memory problem but is noisy: very slow convergence, "zigzag" path
Mini-batch SGD	Uses a small batch (e.g. 100 to 256 records)	Better resource use, better convergence, less noise. **Standard in practice.** Still some noise/oscillation

4.3 Improved optimizers (learn the story: each fixes the previous problem)

Optimizer	Idea	Formulas	Problem left
SGD with Momentum	Reduces noise and smooths descent using **Exponential Weighted Average (EWA)**. Gives weight β to past gradient and (1−β) to current noisy gradient. Acts like **inertia**: noisy up-down gradients cancel, consistent gradients add up, so convergence is quicker	$v_{dw} = \beta\ v_{dw}^{(t-1)} + (1-\beta)\ \partial L/\partial W$ $W^{(t)} = W^{(t-1)} - \eta\ v_{dw}$	Learning rate η is still **fixed** for all parameters
AdaGrad	Changes learning rate **adaptively** per parameter. Divides η by the root of the **cumulative sum of past squared gradients**	$\eta_{ada} = \eta\ /\ \sqrt{(\alpha_t + \varepsilon)}$ $\alpha_t = \Sigma\ (\partial L/\partial W_i)^2$	α_t keeps growing, so η_{ada} becomes tiny and learning **stops** (slows or halts)
RMSprop	Fixes AdaGrad by using **EWA of squared gradients** (second moment) so the denominator does not grow without limit	$s_{dw} = \beta_2\ s_{dw}^{(t-1)} + (1-\beta_2)(\partial L/\partial W)^2$ $W^{(t)} = W^{(t-1)} - (\eta\ /\ \sqrt{(s_{dw} + \varepsilon)}) \cdot \partial L/\partial W$	Uses the noisy current gradient; **lacks smoothing** of Momentum
Adam	**Momentum + RMSprop** together, with bias correction	See below	Best overall

4.4 Adam (Adaptive Moment Estimation)

- One of the most widely used optimizers. Combines **Momentum** and **RMSprop**, and gives an **adaptive learning rate for each parameter**.
- Uses exponentially decaying averages of past gradients, called **moment estimates**:
  - **First moment (v_{dw}):** EWA of the **gradient** (Momentum part): gives **direction and smoothing**. $v_{dw} = \beta_1\ v_{dw} + (1-\beta_1)\ dW$; β_1 typically **0.9**.
  - **Second moment (s_{dw}):** EWA of the **squared gradient** (RMSprop part): gives **adaptive step size** $1/\sqrt{s_{dw}}$. $s_{dw} = \beta_2\ s_{dw} + (1-\beta_2)(dW)^2$; β_2 typically **0.999**. Parameters with large recent gradients get a smaller learning rate, and vice versa.
- **Bias correction:** the hat versions ($\hat{v}_{dw}$, $\hat{s}_{dw}$) are bias-corrected values, so the initial steps are accurate.

> **Adam update rule:** $W := W - \alpha \cdot \hat{v}_{dw}\ /\ (\ \sqrt{\hat{s}_{dw}} + \varepsilon\ )$
> W = weight; α = master learning rate; $\hat{v}_{dw}$ = direction/smoothing (Momentum); $1/\sqrt{\hat{s}_{dw}}$ = adaptive step size (RMSprop); ε = tiny number (e.g. 10^{-8}) to avoid division by zero.

> **Conclusion:** Adam is the most efficient because both the **smoothing factor** and the **adaptive learning rate** are applied together, giving fast and stable convergence.

5. Regularization

- **Regularization** = techniques to **prevent overfitting** and improve **generalization** on unseen data.
- **Overfitting:** model does very well on training data but poorly on new (test/real-world) data. Causes: model too complex, too many parameters, limited training data, memorizing instead of learning patterns.
- **Importance:** better accuracy on unseen data, stable training, less sensitivity to noise, robust features.

5.1 Dropout

- Simple but powerful, invented by **Geoffrey Hinton**.
- During **training**, a Dropout layer randomly **sets a fraction of neurons to zero** at each update step. With dropout rate 0.5, about half the neurons in that layer are ignored in each iteration: they take no part in the forward pass or backpropagation.
- **Intuition (ensemble effect):** the network cannot depend on any single neuron, so it learns **robust, redundant** features and avoids co-adaptation between neurons. Like training many smaller networks at once.
- **Inference (testing):** dropout is **used only during training**. At test time all neurons are active, but the weights are scaled by the keep factor (e.g. × 0.5 for a 50% rate) so the expected output matches training.
- **Example:** in a CNN's fully connected layer, one neuron may specialize in a noisy feature (a tiny scratch on a lens). With dropout that neuron is sometimes off, so other neurons learn general features and the test result improves.

5.2 Batch Normalization (Batch Norm)

- Normalizes the **inputs to a layer across a mini-batch** during training. Stabilizes learning, speeds up training, and reduces sensitivity to initialization.
- **Problem solved: Internal Covariate Shift (ICS).** As earlier layers' parameters change during training, the **distribution of inputs to later layers keeps changing**. Later layers must keep re-adapting, which slows training and needs lower learning rates.
- **Where:** inserted between a hidden layer (Dense or Conv2D) and its **activation function**.

Two steps (for mini-batch of size m)

> **1. Normalization** (mean 0, variance 1 across the mini-batch):
> $\hat{x}^{(i)} = (x^{(i)} - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)}$
> μ_B = mini-batch mean; σ_B^2 = mini-batch variance; ε = small constant (e.g. 10^{-7}) to avoid division by zero.
> **2. Scale and shift** (learned parameters γ and β):
> $BN(x^{(i)}) = \gamma \hat{x}^{(i)} + \beta$

γ (scaling) and β (shifting) let the network **learn the best distribution** for the layer inputs instead of being fixed at mean 0 and variance 1. This matters because a different mean/variance may suit the next activation (like ReLU) better.

Example

Without Batch Norm, an early conv layer may give skewed, large positive values, pushing the next layer into the **flat (saturated) region of sigmoid/tanh**, so gradients become zero (vanishing gradient).

Benefits

- **Accelerated training:** allows much higher learning rates without diverging.
- **Stabilizes gradients:** keeps inputs to activations in a good range; prevents exploding/vanishing gradients.
- **Reduced regularization need:** mini-batch mean/variance adds slight noise, a weak regularization, sometimes reducing the need for Dropout.
- **Less sensitive to initialization:** less dependent on careful weight initialization.

Dropout	Batch Normalization
Randomly turns off neurons in training	Normalizes layer inputs over each mini-batch
Prevents **overfitting**	Stabilizes and **speeds up training** (also weak regularizer)
Not used at inference (all neurons on)	Uses learned γ and β

6. Distributed Training on GPUs and TPUs

- Large models (LLMs, big CNNs) and terabyte datasets are too much for one CPU/GPU. **Distributed training** cuts training time from months to days or hours.
- **Why needed: model size** (billions of parameters, terabytes of memory); **dataset size** (millions of images, petabytes of text); **time-to-train** (fast experimentation and deployment).

Hardware

Hardware	Architecture	Use
GPU	Highly parallel, thousands of small efficient cores (CUDA cores), optimized for throughput	General parallel computing; widely available (NVIDIA CUDA ecosystem)
TPU	Custom **ASIC** designed by **Google** for tensor operations (matrix multiply/add); large on-chip memory and high-bandwidth interconnect (ICI)	Large-scale neural network training

Parallelism types

Type	How it works	When / notes
Data parallelism	Model is **replicated** on every device. Data is split into non-overlapping mini-batches, each device trains on its own batch. Gradients are **aggregated (averaged)** so all replicas update together	Easiest and most common (CNNs, RNNs). Communication happens during **gradient synchronization**; overhead grows with devices and model size
Model parallelism (sharding)	Model is too big for one device, so it is **split across devices**. Output of one device is input to the next	Needed for huge models (e.g. LLMs). Communication between every layer, forward and backward. **Layer-wise (pipeline):** different layers on different devices. **Intra-layer (tensor parallelism):** one large layer's weights split across devices
Hybrid parallelism	Combine both: model parallelism inside a server/node, data parallelism across nodes	Maximum scalability, for massive (trillion-parameter) models

GPU ecosystem

- **CUDA:** NVIDIA's parallel computing platform and programming model.
- **NCCL:** NVIDIA Collective Communications Library; optimized **All-reduce, All-gather** for fast gradient synchronization in data parallelism.
- **Interconnects: PCIe** connects GPUs in a server (bottleneck for big gradient transfers); **NVLink/NVSwitch** = fast direct GPU-to-GPU link that cuts latency within a node.
- **PyTorch: DistributedDataParallel (DDP)** is the standard, optimized way to do data parallelism (uses process groups + NCCL). **Horovod** (by Uber) simplifies distributed training across PyTorch/TensorFlow/Keras using **ring-allreduce**.

TPU ecosystem (JAX/TensorFlow)

- **TPU Pods:** supercomputers with hundreds or thousands of TPU cores joined by a high-bandwidth 2D/3D torus **Inter-Chip Interconnect (ICI)**.
- **Synchronous operation:** the compiler optimizes the whole computation graph across all cores before running.
- **JAX/Flax** is preferred for best TPU performance. **pmap** = replicates a function across cores (data parallelism). **pjit** = advanced model + data parallelism; you specify how weights and data are partitioned (sharding). Collective operations are built into the hardware.

Challenges and fixes

Challenge	Solution
Communication bottleneck	**Gradient compression** (32-bit to 16/8-bit before sending); **asynchronous training** (stale gradients, less common now)
Mixed precision training (FP16/BF16)	Use lower-precision numbers: ~2x memory saving, ~2x communication speedup, and hardware acceleration (Tensor Cores on NVIDIA, MXU on TPU)
Large batch training	Many devices make the total batch very large and hurt convergence/generalization; **LARS** and **LAMB** optimizers adjust the learning rate per layer for stable large-batch training

Case study: sharding LLMs (uses all techniques)

Technique	Goal	Application
Data parallelism	Speed up training	Replicate the full model across server racks (nodes)
Tensor parallelism	Split huge weight matrices	Split large linear/attention layers across devices within one server
Pipeline parallelism	Split the model's layers	Different transformer layers on different device groups (pipeline stages)

7. Last-Minute Revision

- **TensorFlow 2.x:** Google; eager execution, Keras API, AutoGraph, Serving/Lite/JS; best for production. **PyTorch:** Meta; dynamic graph, easy debugging, best for research and HuggingFace.
- **Gradient clipping:** stops exploding gradients (NaN loss). Value-based: cap in a range. Norm-based: g × max_norm/||g||. Example: [4,8,12], max 10 gives [2.672, 5.344, 8.016].
- **LR schedules:** Step (LR×γ), Exponential ($LR_0\gamma^{epoch}$), Polynomial, ReduceLROnPlateau, Cyclical (triangular wave).
- **Iterations = records / batch size.** Epoch = whole dataset once.
- **GD:** $\theta = \theta - \eta\nabla J$. Batch (all data) → SGD (one record, noisy) → Mini-batch (standard) → Momentum (smooth, EWA) → AdaGrad (adaptive, but LR dies) → RMSprop (EWA of squared gradients) → **Adam** (Momentum + RMSprop + bias correction).
- **Dropout:** randomly drops neurons in training only; fights overfitting. **Batch Norm:** normalizes layer inputs per mini-batch, then $\gamma\hat{x} + \beta$; fixes internal covariate shift, allows higher learning rates.
- **Data parallelism** = copy model, split data, average gradients. **Model parallelism** = split model across devices. **Hybrid** = both. GPU tools: CUDA, NCCL, NVLink, DDP, Horovod. TPU tools: ICI, pods, JAX pmap/pjit.
- **Mixed precision** (FP16/BF16) saves memory and bandwidth; **LARS/LAMB** for large batches.